

WebSocket con STOMP

El canal de actividad en vivo

¿Cómo se entera un cliente de algo que acaba de pasar?

Con REST puro, el cliente siempre pregunta primero: manda una petición, el servidor responde, y ahí termina esa conversación.

¿Qué pasa cuando el servidor sabe algo que al cliente le interesa, y el cliente todavía no ha preguntado por ello? Con petición-respuesta puro, el servidor solo puede reaccionar, nunca tomar la iniciativa.

Las soluciones históricas, en orden de aparición

1. Recargar la página — a mano, o con un `setTimeout`. Brutal y lento: el usuario ve parpadear toda la página por un dato que probablemente ni ha cambiado.
2. Polling: el cliente pregunta cada X segundos, ¿ha cambiado algo? Funciona, pero con coste real: peticiones constantes, casi siempre "no, nada nuevo", y retraso de hasta X segundos.
3. Long polling: el servidor retiene la respuesta hasta que hay algo que contar. Mejora el retraso, pero sigue siendo, en el fondo, peticiones repetidas.

Las tres dan vueltas al mismo obstáculo: el cliente sigue siendo quien pregunta. WebSocket elimina la necesidad de preguntar, en vez de perfeccionar la pregunta.

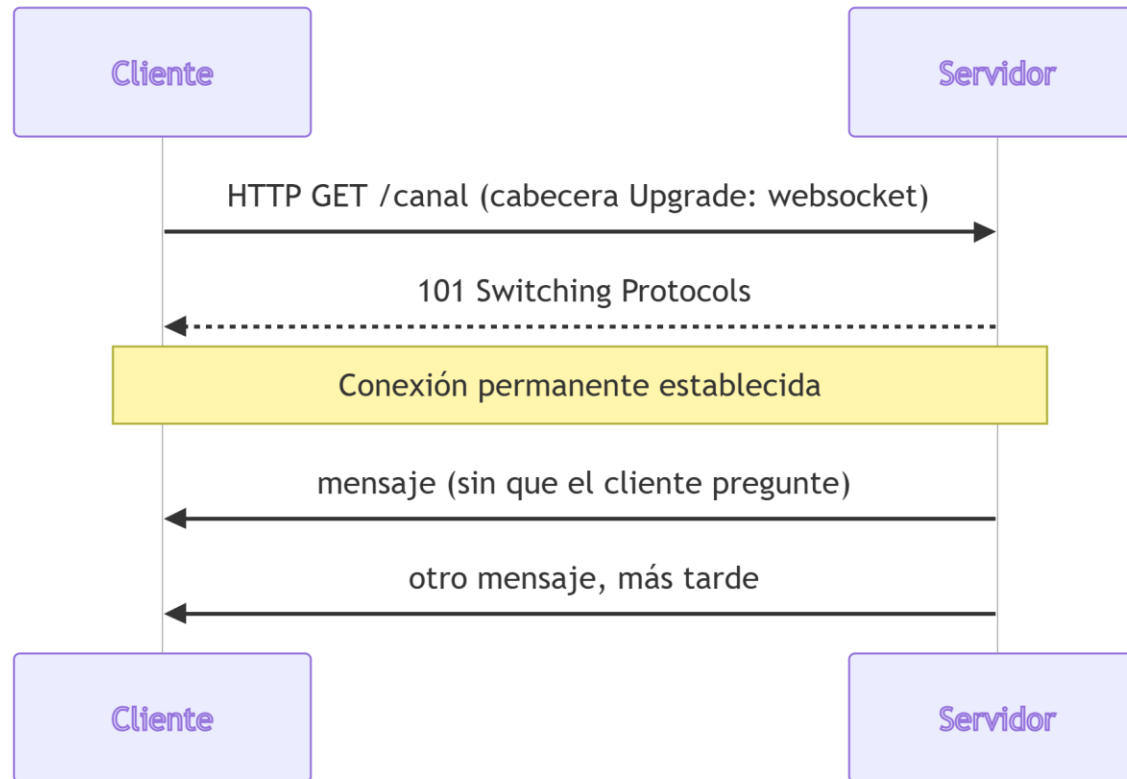
WebSocket: el canal bidireccional persistente

WebSocket implementa el tercer modelo de comunicación (visto en el apartado anterior) mediante un protocolo bidireccional sobre una conexión TCP que permanece abierta mientras ambos extremos la mantienen.

Cliente y servidor pueden enviar mensajes en cualquier momento, sin abrir una conexión nueva para cada intercambio. No es el mismo API que el Socket de Java clásico, aunque por debajo siga existiendo una conexión TCP.

Del HTTP normal al canal permanente

El navegador manda una única petición HTTP con una cabecera especial (Upgrade) que pide cambiar de protocolo. Si el servidor acepta, la conexión no se cierra: se queda abierta de forma permanente.



El cliente no se queda esperando: el callback

El servidor manda mensajes sin que nada los haya provocado desde el cliente. Y el cliente, en su lado, tampoco se queda bloqueado esperándolos.

El cliente deja registrado un aviso —un callback— y sigue a lo suyo; en cuanto llega un mensaje por ese canal, se ejecuta ese aviso automáticamente, tantas veces como mensajes lleguen. Es el mismo patrón `onConnect/subscribe` de la Actividad 4.2.

Qué es STOMP

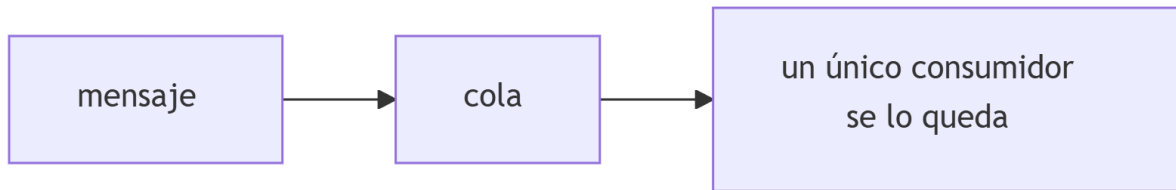
WebSocket por sí solo no define conceptos como destinos, suscripciones o tipos de aviso. STOMP (Simple Text Oriented Messaging Protocol) añade esa capa encima:

- Destino: un nombre para cada tipo de mensaje, como /topic/notas o /topic/alertas-stock.
- Suscripción: cada cliente se suscribe solo a los destinos que le interesan, sin que le lleguen los demás.

El mismo patrón de publicación-suscripción que ya conoces de `ApplicationEventPublisher` (Tema 3), ahora con el suscriptor fuera de la JVM, en el navegador.

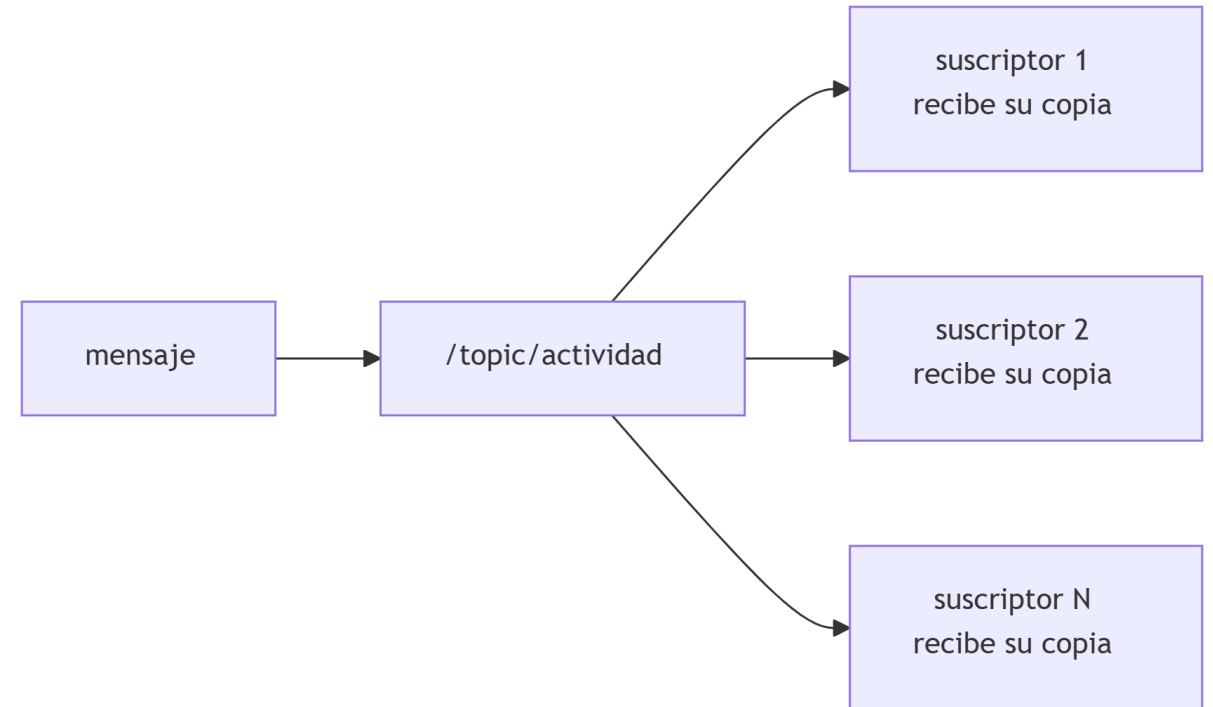
El reparto de mensajes: distinto al de RabbitMQ

Cola de RabbitMQ (Tema 3)



Un destino STOMP puede tener varios suscriptores a la vez, y todos reciben su propia copia. En una misma cola de RabbitMQ, cada mensaje se entrega a uno de sus consumidores.

Topic de STOMP

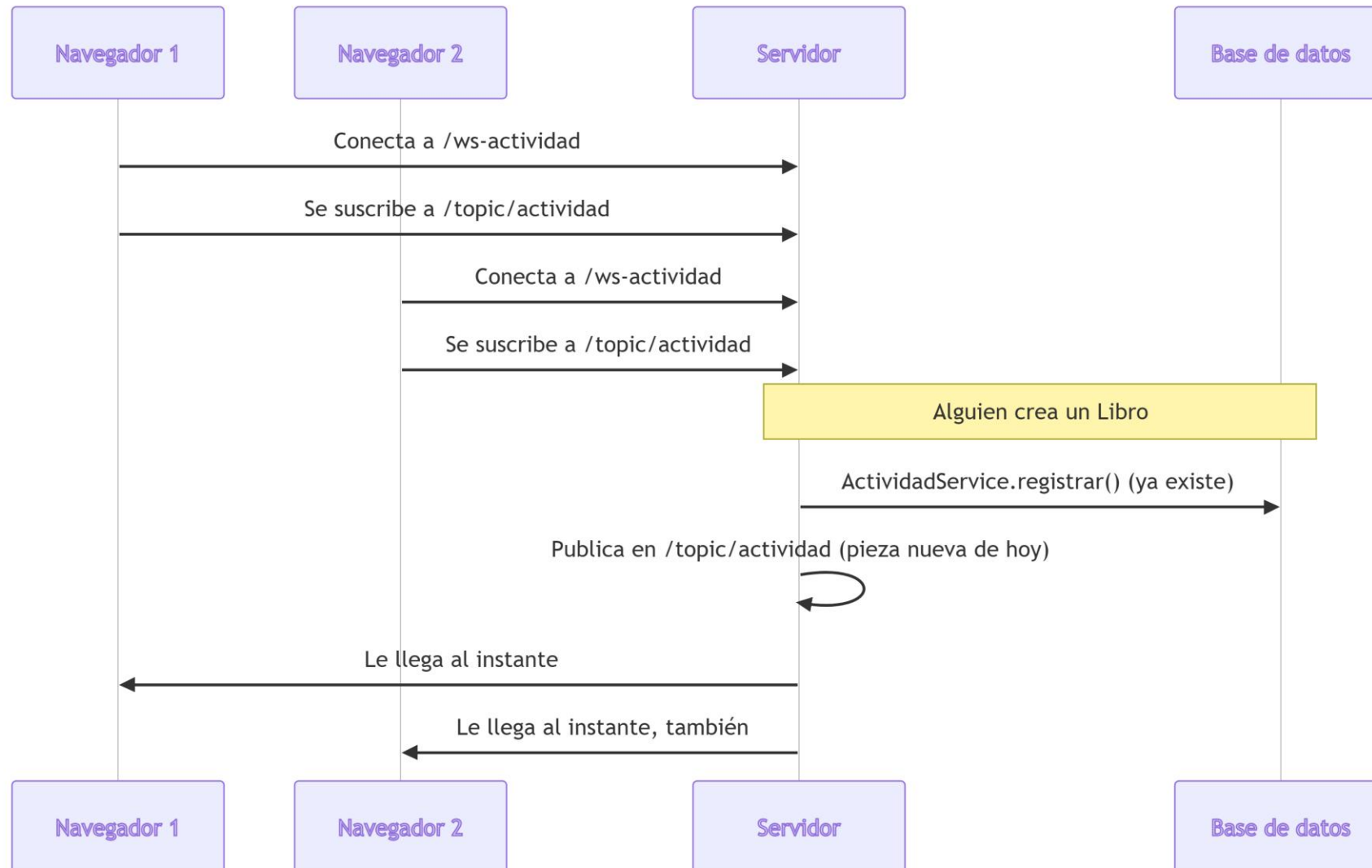


El caso de uso: actividad en vivo

Cada vez que se crea, modifica o borra un Libro, `ActividadService.registrar()` ya guarda ese registro en la base de datos — hoy solo puede consultarse con `GET /api/v1/actividad`, en frío, y solo por ADMIN.

La pieza que falta: publicar esos mismos registros en vivo, según ocurren, a quien esté mirando en ese momento.

La foto completa de lo que vas a construir



Las tres piezas base de la configuración

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-actividad").setAllowedOriginPatterns("*");
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic");
        registry.setApplicationDestinationPrefixes("/app");
    }
}
```

Activar el soporte de mensajería WebSocket, declarar a qué URL se conecta el cliente, y decidir cómo se reparten los mensajes: las tres caben en una única clase de configuración.

Qué hace cada método

Método	Responsabilidad
@EnableWebSocketMessageBroker	Activa el soporte de mensajería WebSocket con STOMP sobre toda la aplicación
registerStompEndpoints	Declara la URL a la que el cliente se conecta inicialmente — el /ws-actividad
configureMessageBroker	Activa el broker (mismo concepto que en RabbitMQ) y decide qué prefijos gestiona: /topic y /app

Aquí no hace falta ningún @RestController

registerStompEndpoints ya declara el endpoint — a diferencia de REST, no hace falta @GetMapping. El @MessageMapping("/app") sí es el equivalente de @RequestMapping, pero solo para mensajes que llegan después de que la conexión ya esté establecida.

Dónde aparece el destino concreto, y quién lo pone

El código de configuración solo declara el prefijo /topic, nunca /topic/actividad. El destino concreto aparece en otros dos sitios:

```
// Servidor (Actividad 4.2)
messagingTemplate.convertAndSend("/topic/actividad", "Nuevo Libro");
```

```
// Cliente
client.subscribe('/topic/actividad', function (mensaje) {
    console.log('Ha llegado un aviso:', mensaje.body);
});
```

Publicador y suscriptor tienen que escribir el mismo nombre, carácter a carácter, para encontrarse — no se declara en un sitio central.

La diferencia clave con RabbitMQ

	RabbitMQ (Tema 3)	STOMP
Qué declaras en la configuración	Cada cola, con su propio @Bean	Solo el prefijo ("/topic")
Dónde aparece el nombre concreto	En la propia declaración de la cola	Solo donde se publica y donde se suscribe
Si publicador y receptor no coinciden	El binding y la routing key deciden si llega; si no encajan, puede quedar sin enrutar	El mensaje se publica en otro destino y ese suscriptor no lo recibe

Dos mundos distintos, y dónde vive cada línea



- Java (`WebSocketConfig`, `messagingTemplate.convertAndSend`) vive en tu proyecto Spring: se compila con Maven, mismo servidor de siempre.
- JavaScript del cliente no lo ejecutas tú: es un fichero en `src/main/resources/static/` que ejecuta el navegador de quien lo visita, sin compilar nada.

Dónde escribes tú esa línea, en tu propio proyecto

```
@Service
@RequiredArgsConstructor
public class NotaLecturaService {
    private final NotaLecturaRepository notaLecturaRepository;
    private final SimpMessagingTemplate messagingTemplate; // pieza nueva

    public NotaLectura crear(NotaLecturaCreatedDTO dto) {
        NotaLectura guardada = notaLecturaRepository.save(mapToEntity(dto));
        messagingTemplate.convertAndSend("/topic/notas", guardada); // pieza nueva
        return guardada;
    }
}
```

SimpMessagingTemplate se inyecta por constructor igual que cualquier repositorio o servicio ya conocido — el mismo método que guardaba en base de datos, ahora también publica por WebSocket.

Dos preguntas frecuentes

¿El cliente recibe el objeto guardada tal cual?

No. `convertAndSend(destino, objeto)` lo convierte primero a JSON, igual que Spring hace con cualquier respuesta REST. En JavaScript, `mensaje.body` es una cadena de texto: hace falta `JSON.parse(mensaje.body)` antes de leer sus campos.

¿En qué hilo se ejecuta ese `convertAndSend`?

En tu proyecto real, `registrar(...)` lo llama el consumer de RabbitMQ (Tema 3), desde uno de sus hilos de escucha, no desde el hilo de Tomcat que atendió la petición original. Spring distribuye el mensaje mediante su propia infraestructura de mensajería.

Comparado con lo que ya conoces

	REST	WebSocket
Quién inicia	El cliente, cada vez	El cliente, una vez (la conexión inicial)
Respuestas por petición	Una	Ninguna fija — el servidor envía cuando quiere
Duración de la conexión	Corta, se cierra tras la respuesta	Persistente
Caso de uso típico	Consultas bajo demanda	Notificaciones en tiempo real

La comparación conceptual más directa sigue siendo con `ApplicationEventPublisher`, no con `RabbitMQ`: ambos siguen un modelo de publicación-suscripción. Y la concurrencia de los navegadores la gestiona Spring, no tú — aquí no escribes ningún `Thread`.

Ideas clave

- El problema de fondo: con petición-respuesta puro, el cliente siempre pregunta primero. Recargar, polling y long polling son soluciones parciales al mismo problema.
- WebSocket es el canal bidireccional persistente: petición HTTP inicial con cabecera Upgrade, después conexión permanente donde el servidor envía sin que el cliente pregunte, y el cliente recibe por callback.
- STOMP añade semántica de mensajería (destinos, suscripciones) sobre WebSocket.
@EnableWebSocketMessageBroker + registerStompEndpoints + configureMessageBroker son las tres piezas de configuración base.
- Mismo modelo pub-sub que ApplicationEventPublisher (Tema 3), pero con el suscriptor fuera de la JVM. En una cola RabbitMQ varios consumidores compiten; en un topic STOMP todos los suscriptores reciben cada publicación.